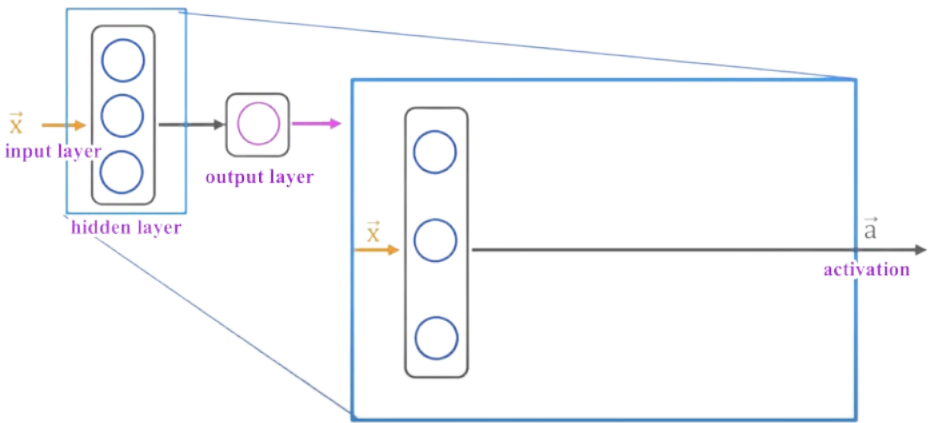


- ▼ 神经元层
 - 输入层 (Input Layer)
 - 隐藏层 (Hidden Layer)
 - 输出层 (Output Layer)
- ▼ 前向传播 (Forward Propagation)
 - ▼ TensorFlow 框架
 - 矩阵
 - 简单网络
 - 单网络实现
- ▼ 激活函数
 - ▼ ReLu 激活函数
 - Sigmoid 缺点:
 - ReLU 函数
 - 线性激活函数
 - ▼ Softmax 激活函数
 - 多类分类 (Multiclass Classification)
 - Softmax 函数定义
 - 多类别代价函数 (Categorical Cross-Entropy Loss)
 - 代码片段
- ▼ 高级优化
 - ▼ Adam 优化算法 (Adaptive Moment Estimation)
 - Adam 证明 (简化版)
 - 代码片段
- ▼ 网络类别
 - 全连接 / 密集 (Fully Connected Layer / Dense Layer)
 - 卷积 (Convolutional Layer / Conv2D)
- ▼ 模型评估
 - ▼ 性能
 - ▼ 线性回归
 - 训练阶段 (Fit Parameters)
 - 评估 (Compute Test/Train Error)
 - ▼ 分类问题
 - 训练阶段
 - 评估
 - ▼ 训练交叉验证测试集
 - 交叉验证集 (Cross Validation Set)
 - 偏差与方差
 - 正则化
 - 学习曲线
 - ▼ 技巧补充
 - 错误分析
 - Data Augmentation
 - 迁移学习
 - ▼ 倾斜训练集 (Skewed Datasets)
 - 混淆矩阵 (Confusion Matrix)
 - 精准率 (Precision)
 - 召回率 (Recall) / 敏感性 (Sensitivity)
 - F1 分数 (F1 Score)
 - Precision-Recall 权衡
- ▼ 决策树
 - ▼ Recursion 二叉分裂
 - 熵 (Entropy)
 - 信息增益 (Information Gain)
 - 停止条件
 - Continuous Variable
 - 回归问题

- ▼ 集成树 (Ensemble Methods)
 - 放回抽样
 - 随机森林
 - XGBoost 优化

神经元层

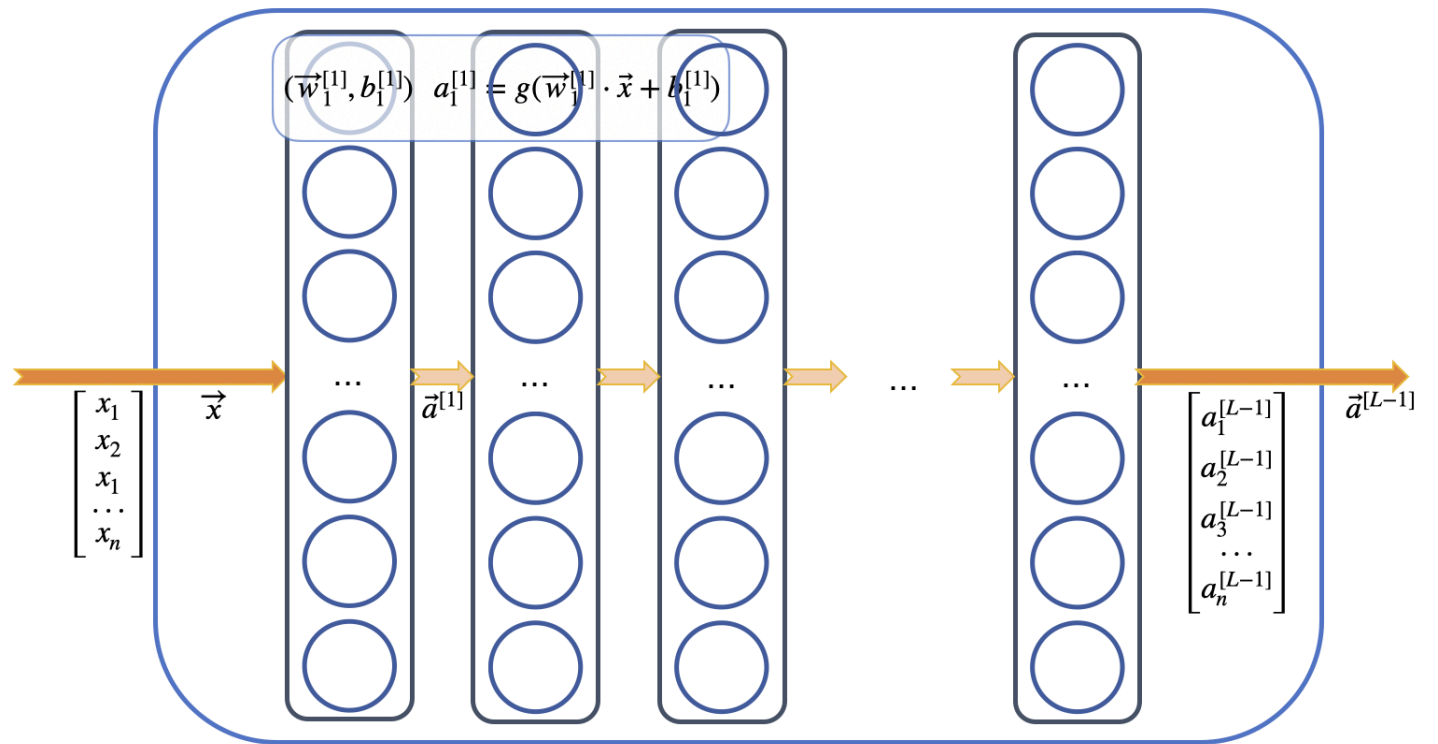
在 Neural Network [1]中，一组水平排列的神经元，它们收到来自前一层的输入，并通过自己的激活与计算函数得到输出，并将这些输出作为下一层的输入。



输入层 (Input Layer)

- 位置：Neural Network 的最前端，收到原始 data。
- 功能：它不执行任何激活与计算函数，仅仅是 data 的入口。
- 节点数：等于输入特征数 (n)。
- 表示：通常用 $\vec{a}^{[0]}$ 来表示 Input Layer 的激活值（即 $\vec{x}$ ）。

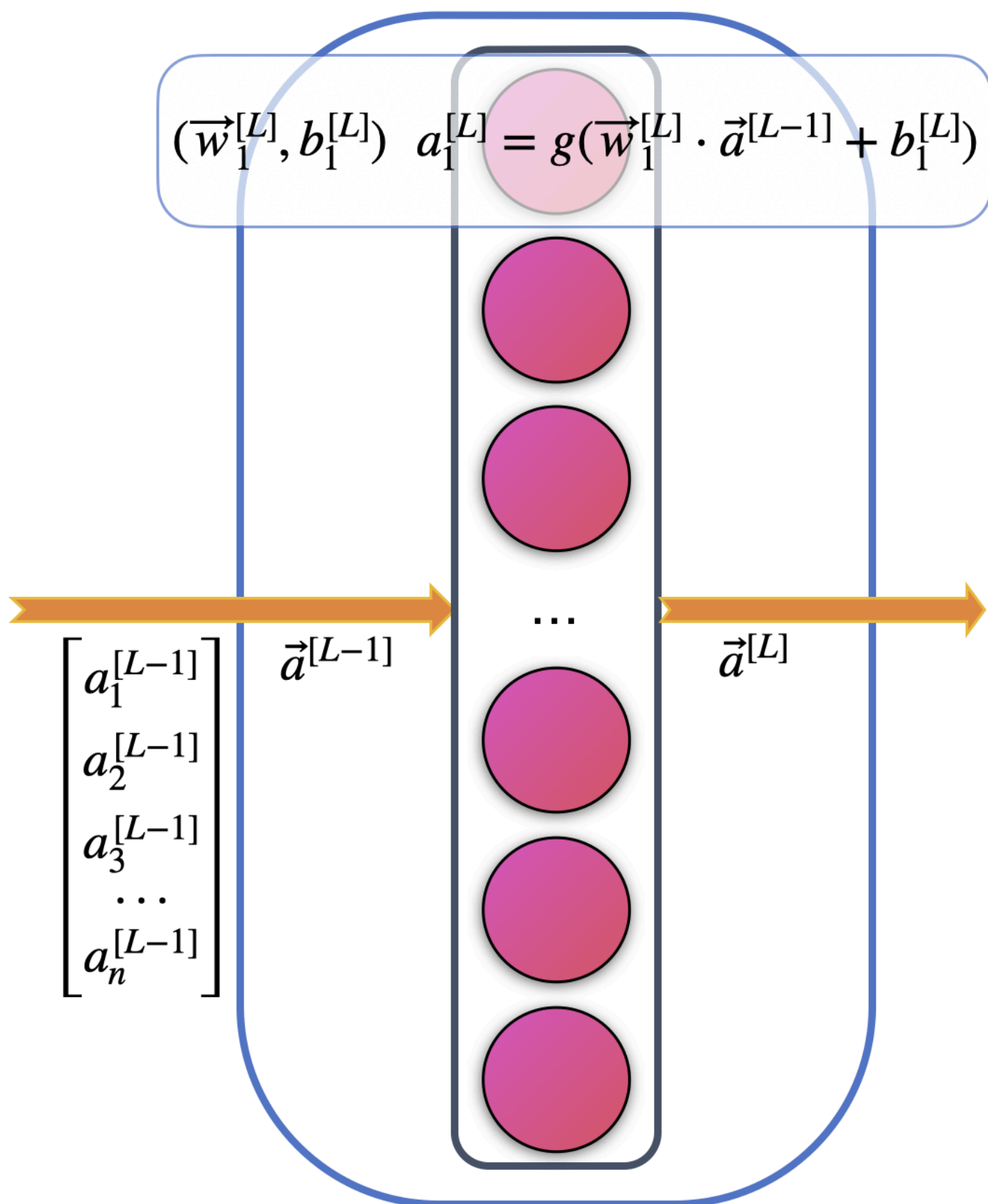
隐藏层 (Hidden Layer)



- 位置：位于 Input Layer 和 Output Layer 之间，一个 Neural Network 可以有一个或多个 Hidden Layer。
- 功能：进行计算和自动学习特征，Hidden Layer 的所有神经元均会对收到的输入进行加权求和，然后运行对应的非线性激活函数。

- 节点数: Hidden Layer 中的节点数是 Neural Network 的一个超参数。
- 表示: 通常用 $\vec{a}^{[l]}$ 来表示 l_{th} 个 Hidden Layer 的激活值。

输出层 (Output Layer)



- 位置: Neural Network 的最末端, 得到模型的最终预测。
- 功能: 根据分类或回归, Output Layer 会得到不同的结果。
- 节点数:
 - 二分类问题: 通常只有一个神经元, 采用 Sigmoid 激活函数, 输出一个 0 到 1 之间的概率。

- 多分类问题：多个神经元，采用 Softmax 激活函数，输出每个类别的概率分布。
- 回归问题：通常只有一个神经元，无激活函数（或线性激活函数），输出预测值。
- 表示：通常用 $\vec{a}^{[L]}$ 来表示 Output Layer 的激活值。

前向传播 (Forward Propagation)

前向传播是 Neural Network 在给定输入的 data 和当前模型的参数（ $\vec{w}$ 和 $\vec{b}$ ）的情况下，从 Input Layer 开始，逐步计算并传输激活值，直到 Output Layer 生成最终预测结果的过程。

TensorFlow 框架

矩阵

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

在 *TensorFlow* 中存储一个矩阵如下：

```
x = np.array([[1, 2, 3],
              [4, 5, 6]])
```

并且行向量和列向量的实现方式也得采取“双括号”的形式，以证明其为矩阵而不是单纯的一维表，比如 $\begin{bmatrix} 200 & 17 \end{bmatrix}$ 以及 $\begin{bmatrix} 200 \\ 17 \end{bmatrix}$ 实现方式如下：

```
x = np.array([[200, 17]])
x = np.array([[200],
              [17]])
```

简单网络

```
x = np.array([[200.0, 17.0]])
layer_1 = Dense(units = 3, activation='sigmoid')
a_1 = layer_1(x)

layer_2 = Dense(units = 1, activation='sigmoid')
a_2 = layer_2(a_1)

if a_2 >= 0.5:
    y_hat = 1
else:
    y_hat = 0
```

或者采用 *TensorFlow* 的顺序框架简化代码：

```
model = Sequential([
    Dense(units = 25, activation='sigmoid')
    Dense(units = 15, activation='sigmoid')
    Dense(units = 1, activation='sigmoid')])

model.compile(...)

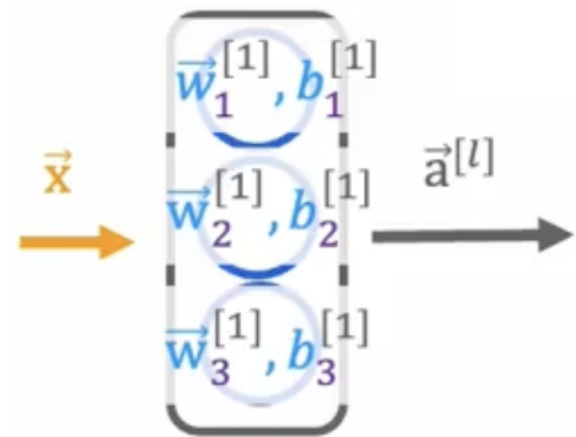
x = np.array([[0, ..., 245, ..., 17],
              [0, ..., 200, ..., 184]])
y = np.array([1, 0])

model.fit(x, y)

model.predict(x_new)
```


单网络实现

前面介绍了借助 TensorFlow 即可 5 行实现一个小的 Neural Network，然而我们显然不能止步于此，让我们自己来实现一个单网络模块吧！



首先我们定义一些参数：

$$\vec{w}_1^{[1]} = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \quad \vec{w}_2^{[1]} = \begin{bmatrix} -3 \\ 4 \end{bmatrix} \quad \vec{w}_3^{[1]} = \begin{bmatrix} 5 \\ -6 \end{bmatrix}$$
$$b_1^{[l]} = -1 \quad b_2^{[l]} = 1 \quad b_3^{[l]} = 2$$

```
W = np.array([
    [1, -3, 5],
    [2, 4, -6]
])
b = np.array([-1, 1, 2])
a_in = np.array([-2, 4])

def dense(a_in, W, b, g):
    # col = 3
    units = W.shape[1]
    a_out = np.zeros(units)
    for j in range(units):
        w = W[:, j]
        z = np.dot(w, a_in) + b[j]
        a_out[j] = g(z)
    return a_out
```

The columns of W represent the number of neurons in the current layer, and the number of rows represents the number of neurons in the previous layer. Therefore, multiply an $m \times n$ matrix by an $n \times 1$ matrix results in an $m \times 1$ matrix, which represents the activation values of the current layer.

为此，以上代码也可以写成矩阵乘法的形式：

```
B = np.array([[-1, 1, 2]])
A_In = np.array([[-2, 4]])
def dense(A_In, W, B, g):
    Z = np.matmul(A_In, W) + B # Z = A_In @ W + B
    A_Out = g(Z)
    return A_Out
```

激活函数

ReLU 激活函数

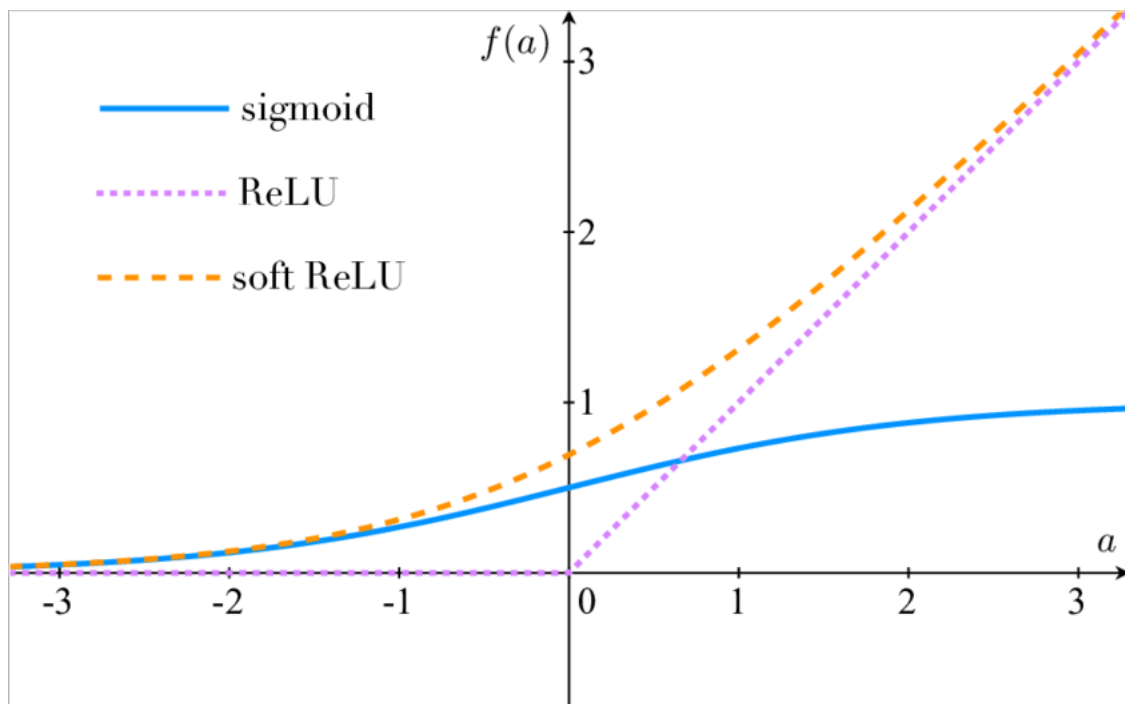
Sigmoid 缺点：

- **Vanishing Gradient**: 当输入 z 的值非常大或非常小时，其导数会非常接近于 0。这意味着在反向传播过程中，gradient 会变得极小，导致权重更新非常慢，甚至停止学习，尤其是网络较深中。

- **输出非 0 中心：** Sigmoid 的输出在 0 到 1 之间，是非 0 中心的，可能会降低 Gradient Descent 的效率。

ReLU 函数

$$g(z) = \max(0, z)$$



优点：

- 当 $z > 0$ 时，导数恒为 1，Gradient Descent 能有效传播。
- **稀疏激活：** 输入负值时输出 0，可以引入稀疏性，有助于降低过拟合。

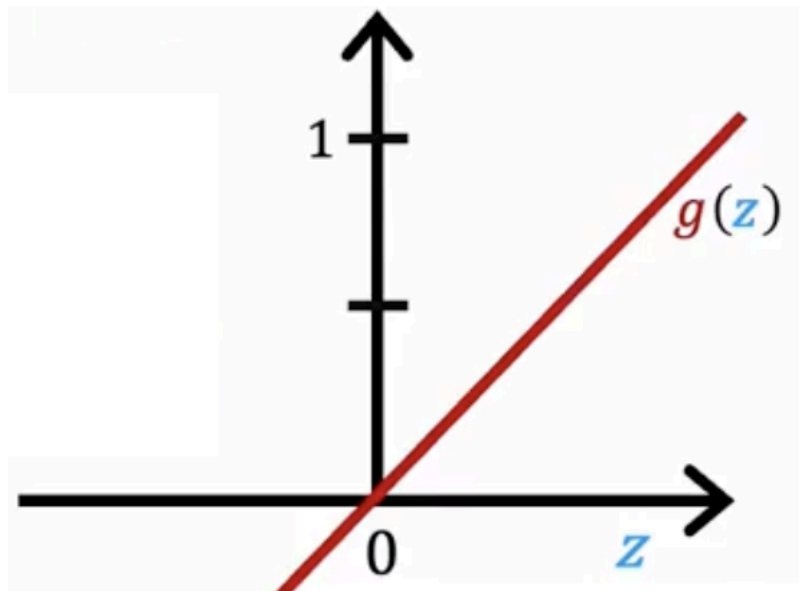
为此，在 Hidden Layer 中，除了部分特殊情况，ReLU 激活函数几乎是所有从业者的掌上明珠。

缺点：

- **“死亡 ReLU”问题 (Dying ReLU)：** 当 z 长时间负值时，它的 Gradient Descent 将永远为 0，该神经元将不再学习（“死亡”）。
- **非 0 中心输出：** 同样，ReLU 的输出也是非 0 中心的。

线性激活函数

当 Output Layer 执行回归问题，且输出值有正有负时，线性激活函数是其首选，此时 Hidden Layer 必须是非线性的，否则模型无法拟合非线性关系；如果输出值非负，也可以采用 ReLU 激活函数。



Softmax 激活函数

多类分类 (Multiclass Classification)

二元分类问题上，Output Layer 通常以 Sigmoid函数为激活函数，而当类别有两个以上的时，称为多类别分类问题，此类问题有一个机制，能给每个类别输出一个概率，且这些概率加起来等于 1。此时，Softmax 函数成了 Output Layer 激活函数的标准。

Softmax 函数定义

Softmax 函数的参数包含一个 K 个任意值的 $\vec{z}$ ，然后将其转换为一个包含 K 个概率值的 $\hat{y}$ 。

对于 Output Layer j_{th} 个神经元的 $z_j = \vec{w_1} \cdot \vec{x} + b$ 值，其 Softmax 激活输出 $\hat{y}_j$ 为：

$$\hat{y}_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}}$$

其中， K 是输出类别数。

多类别代价函数 (Categorical Cross-Entropy Loss)

与二分类的对数损失类似，多类别分类使用交叉熵损失，损失函数如下：

$$\text{loss}(a_1, \dots, a_K, y) = \begin{cases} -\log a_1 & \text{if } y = 1 \\ -\log a_2 & \text{if } y = 2 \\ \vdots & \vdots \\ -\log a_K & \text{if } y = K \end{cases}$$

代码片段

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense

model = Sequential([
    Dense(units=25, activation='relu')
    Dense(units=15, activation='relu')
    Dense(units=10, activation='linear') # Dense(units=10, activation='softmax')
])

from tensorflow.keras.losses import SparseCategoricalCrossEntropy

model.compile(..., loss=SparseCategoricalCroessEntropy(from_logits=True)) # model.compile(loss=SparseCategoricalCrossEntropy())

model.fit(X, Y, epochs=100)

logits = model(X)
f_x = tf.nn.softmax(logits)
```

注：以上代码中未注释版本相较于注释版本可以有效提升舍入精度。

而如果采用独热编码(One-Hot Encoding) 体现真实标签 y (猫: $[1, 0, 0]$, 狗: $[0, 1, 0]$, 鸟: $[0, 0, 1]$) , Softmax 的输出是预测概率 $\hat{y}$ (如, $[0.8, 0.1, 0.1]$) 。此时，当且仅当标签 c 命中时, $y_c^{(i)} = 1$, 其余 $y_j^{(i)} = 0, j \neq c$, 且此时 $\hat{y}_j^{(i)}$ 预测越靠近 1，惩罚越小，越远离 1，代价越大。故代价函数如下：

$$J(W, b) = -\frac{1}{m} \sum_{i=1}^m \sum_{j=1}^K y_j^{(i)} \log(\hat{y}_j^{(i)})$$

高级优化

Adam 优化算法 (Adaptive Moment Estimation)

自适应修正学习率 (α)，妙！实现本节简单介绍一下，了解即可。

Adam 证明（简化版）

对于参数 w_j ：

1. 初始化一阶矩和二阶矩估计：

$v_{dw} = 0, s_{dw} = 0$

2. 迭代：

a. 同前

dw, db

b.

$v_{dw} = \beta_1 v_{dw} + (1 - \beta_1)dw$

$v_{db} = \beta_1 v_{db} + (1 - \beta_1)db$

c.

$s_{dw} = \beta_2 s_{dw} + (1 - \beta_2)(dw)^2$

$s_{db} = \beta_2 s_{db} + (1 - \beta_2)(db)^2$

d. 校正：

$v_{dw}^{corrected} = \frac{v_{dw}}{1 - \beta_1^t}$

$s_{dw}^{corrected} = \frac{s_{dw}}{1 - \beta_2^t}$

e.

$w_j := w_j - \alpha \frac{v_{dw}^{corrected}}{\sqrt{s_{dw}^{corrected} + \epsilon}}$

$b_j := b_j - \alpha \frac{v_{db}^{corrected}}{\sqrt{s_{db}^{corrected} + \epsilon}}$

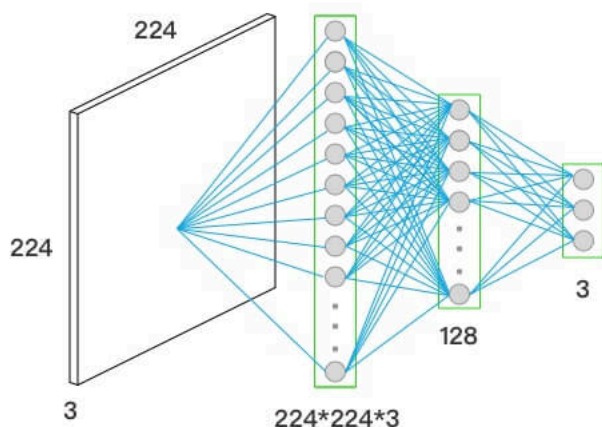
代码片段

```
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
              loss=SparseCategoricalCroessEntropy(from_logits=True))
```

注：采用 Adam 加速，学习率 (α) 可自己摸索。

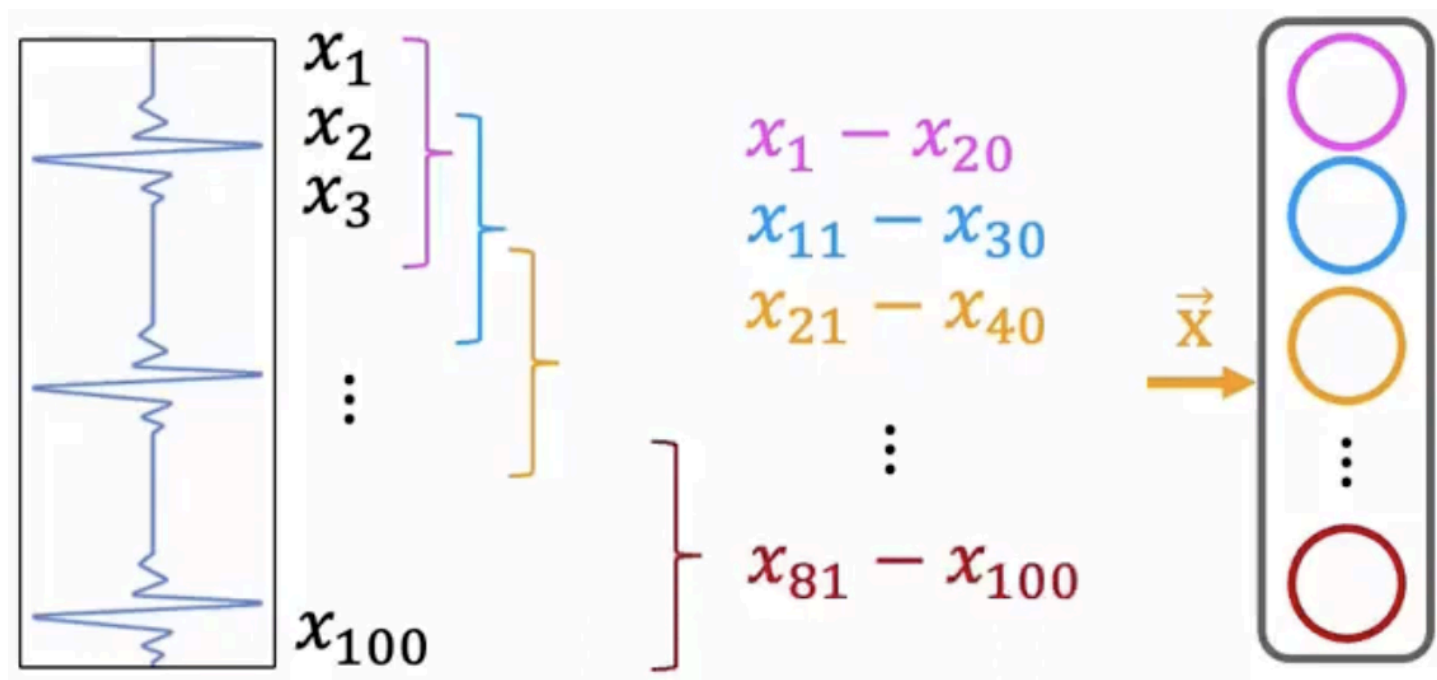
网络类别

全连接 / 密集 (Fully Connected Layer / Dense Layer)



- 功能：前一所有输入与当前的所有神经元完全连接。通常位于其他特殊（如卷积）之后，对提取出的特征实现最终的分类或回归。
- *Keras/TensorFlow* 中： `layers.Dense()`

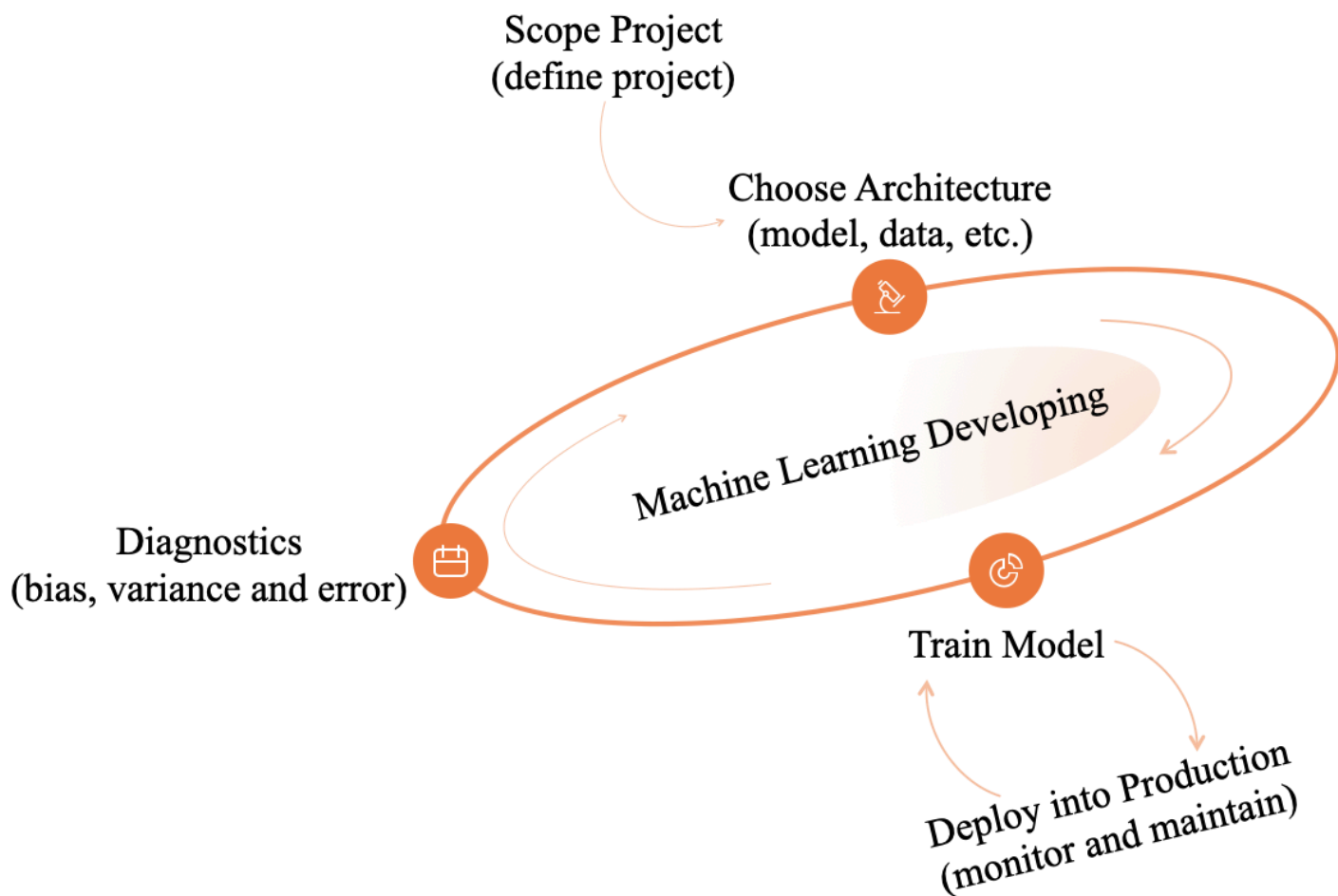
卷积 (Convolutional Layer / Conv2D)



- 功能：卷积神经网络 (CNN) 的核心，特别擅长图像、视频等有网格状结构的 data。它通过滤波 (Filter / Kernel) 在输入上进行滑动和卷积，来提取局部特征。
- *Keras/TensorFlow* 中： `layers.Conv2D()`

模型评估

如下是一个开发 Machine Learning 的系统性流程，所以本节逐一分析其中优化迭代该系统的方法：



性能

为了客观评价模型的性能，我们将所有的 **data**，分为两部分，其中 70% 为训练集 $(x^{(1)}, y^{(1)}), \dots, (x^{(m_{train})}, y^{(m_{train})})$ ，另外 30% 为测试集 $(x_{test}^{(1)}, y_{test}^{(1)}), \dots, (x_{test}^{(m_{test})}, y_{test}^{(m_{test})})$

线性回归

训练阶段 (Fit Parameters)

最小化代价函数 $J(\vec{w}, b)$ 来得到最优参数 $\vec{w}, b$ 。

$$J(\vec{w}, b) = \frac{1}{2m_{train}} \sum_{i=1}^{m_{train}} (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m_{train}} \sum_{j=1}^n w_j^2$$

注：正则化项只在训练时使用，防止训练集过拟合。

评估 (Compute Test/Train Error)

测试 ($J_{test}(\vec{w}, b)$)，评估模型在未见过 **data** 上的性能，不包含正则化项！

$$J_{test}(\vec{w}, b) = \frac{1}{2m_{test}} \sum_{i=1}^{m_{test}} (f_{\vec{w}, b}(\vec{x}_{test}^{(i)}) - y_{test}^{(i)})^2$$

训练 ($J_{train}(\vec{w}, b)$)，评估模型在训练 **data** 上的拟合性，也不包含正则化项！

$$J_{train}(\vec{w}, b) = \frac{1}{2m_{train}} \sum_{i=1}^{m_{train}} (f_{\vec{w}, b}(\vec{x}_{train}^{(i)}) - y_{train}^{(i)})^2$$

由于模型训练是在训练集上，所以训练出来的 $J_{train}(\vec{w}, b)$ 通常较低，除非出现欠拟合，而此时评估性能核心在于 $J_{test}(\vec{w}, b)$ 也达到较低水平。

分类问题

训练阶段

$$\min(J(\vec{w}, b)) = \min(-\frac{1}{m_{train}} \sum_{i=1}^{m_{train}} [y^{(i)} \log(f_{\vec{w}, b}(\vec{x}^{(i)})) + (1 - y^{(i)}) \log(1 - f_{\vec{w}, b}(\vec{x}^{(i)}))] + \frac{\lambda}{2m_{train}} \sum_{j=1}^n w_j^2)$$

评估

对于分类问题，也可以参照线性回归评估 $J_{train}(\vec{w}, b)$ 和 $J_{test}(\vec{w}, b)$ ，但通常采取**分类错误率** (Misclassification Error) 统计。

$$\hat{y} = \begin{cases} 1 & \text{if } f_{\vec{w}, b}(\vec{x}^{(i)}) \geq 0.5 \\ 0 & \text{if } f_{\vec{w}, b}(\vec{x}^{(i)}) < 0.5 \end{cases}$$

错误率：

- count $\hat{y} \neq y$
- 测试错误率 ($J_{test}(\vec{w}, b)$): 测试集中错误分类的样本比例。
- 训练错误率 ($J_{train}(\vec{w}, b)$): 训练集中错误分类的样本比例。

训练交叉验证测试集

假如你有多个模型（以回归模型说明）：

$$\begin{aligned} d = 1 : \quad & f_{\vec{w}, b}(x) = w_1 x + b \\ d = 2 : \quad & f_{\vec{w}, b}(x) = w_1 x + w_2 x^2 + b \\ & \dots \\ d = 10 : \quad & f_{\vec{w}, b}(x) = w_1 x + w_2 x^2 + \dots + w_{10} x^{10} + b \end{aligned}$$

如果只有训练集和测试集，模型用**训练集**训练这 10 个模型，得到不同的参数 $(\vec{w}^1, b^1), \dots, (\vec{w}^{10}, b^{10})$ 。然后，评估每个模型在**测试集**上的 $J_{train}(\vec{w}, b)$ 和 $J_{test}(\vec{w}, b)$ ，最后，选 $J_{test}(\vec{w}, b)$ 最小的那个模型。

此时，如果你反复用**测试集**来选模型或修正超参数，那么模型实际上就是在“**适应**”**测试集**。这样一来，测试集上的 J_{test} 就会成为一个**过于乐观**的泛化误差估计

交叉验证集 (Cross Validation Set)

为此，我们将 **data** 划分成三个独立的部分：

训练集 (Training Set): 约占 **60%**，训练模型的参数 $\vec{w}, b$ ，并测试 J_{train} 。

$$J_{train}(\vec{w}, b) = \frac{1}{2m_{train}} \sum_{i=1}^{m_{train}} (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)})^2$$

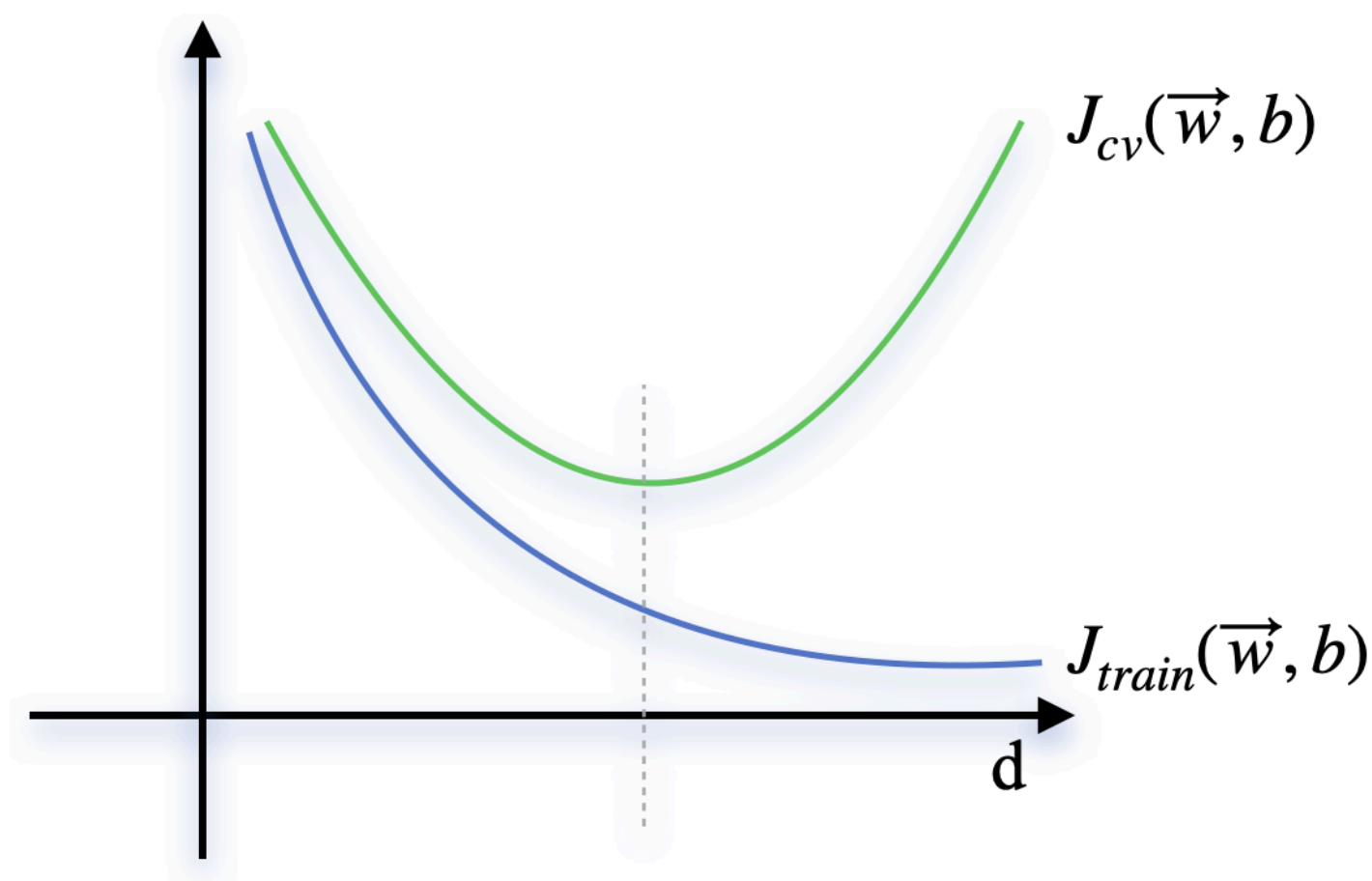
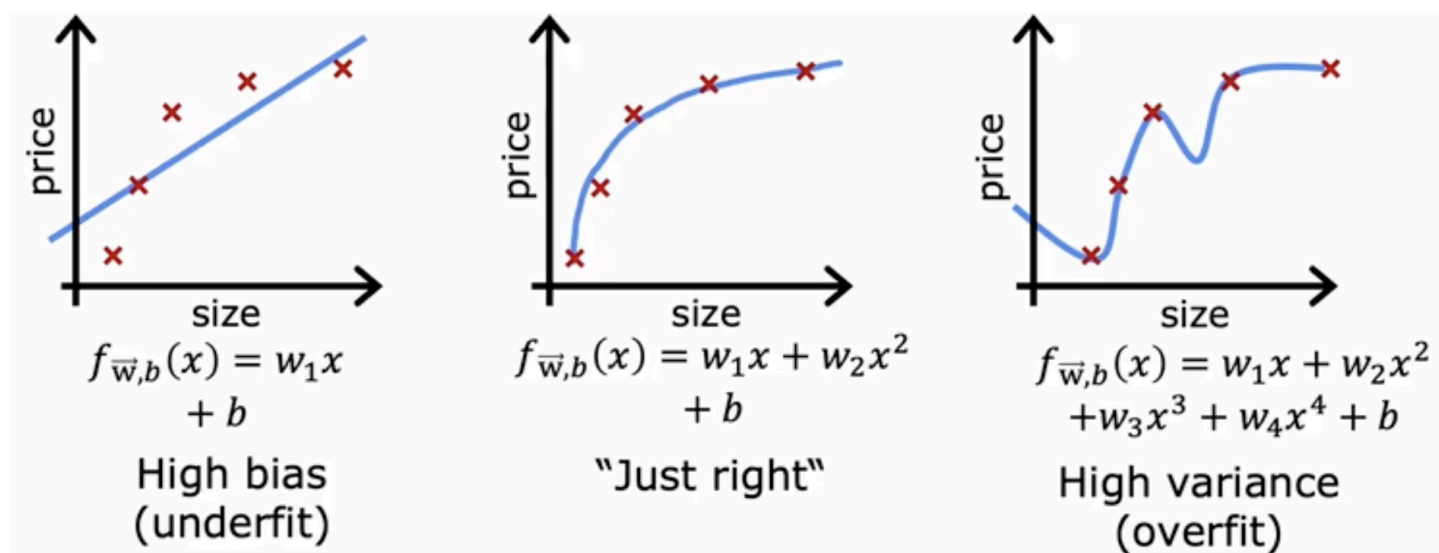
验证集 (Cross Validation Set): 约占 **20%**，选择最佳的，即 $J_{cv}(\vec{w}, b)$ 最小的模型结构或超参数。

$$J_{cv}(\vec{w}, b) = \frac{1}{2m_{cv}} \sum_{i=1}^{m_{cv}} (f_{\vec{w}, b}(\vec{x}_{cv}^{(i)}) - y_{cv}^{(i)})^2$$

测试集 (Test Set): 约占 **20%**，最终评估所选模型的**真实泛化能力**。

$$J_{test}(\vec{w}, b) = \frac{1}{2m_{test}} \sum_{i=1}^{m_{test}} (f_{\vec{w}, b}(\vec{x}_{test}^{(i)}) - y_{test}^{(i)})^2$$

偏差与方差



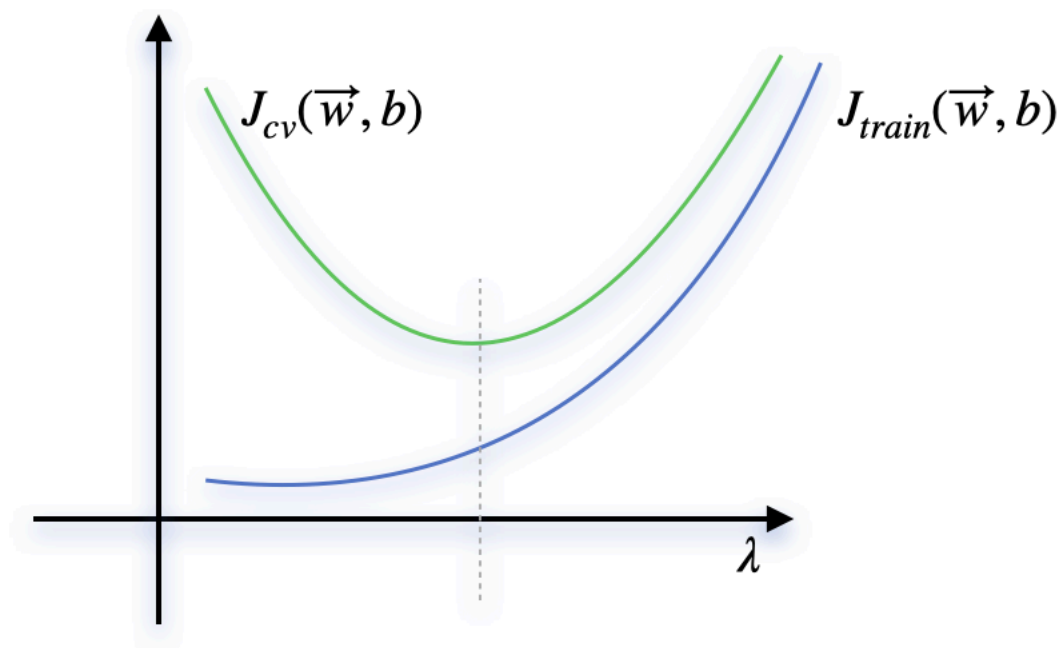
左侧部分 (high bias 区域), J_{train} 会很高, J_{cv} 也会很高, 并且 $J_{cv} \approx J_{train}$, 欠拟合、高偏差。

右侧部分 (high variance 区域), J_{train} 会很低, J_{cv} 会很高, 并且 $J_{cv} \gg J_{train}$, 过拟合、高方差。

有时还会出现高偏差 和 高方差并存 (High Bias and High Variance), 该情况不太常见, J_{train} 高, J_{cv} 很高, 并且 $J_{cv} \gg J_{train}$ 。

正则化

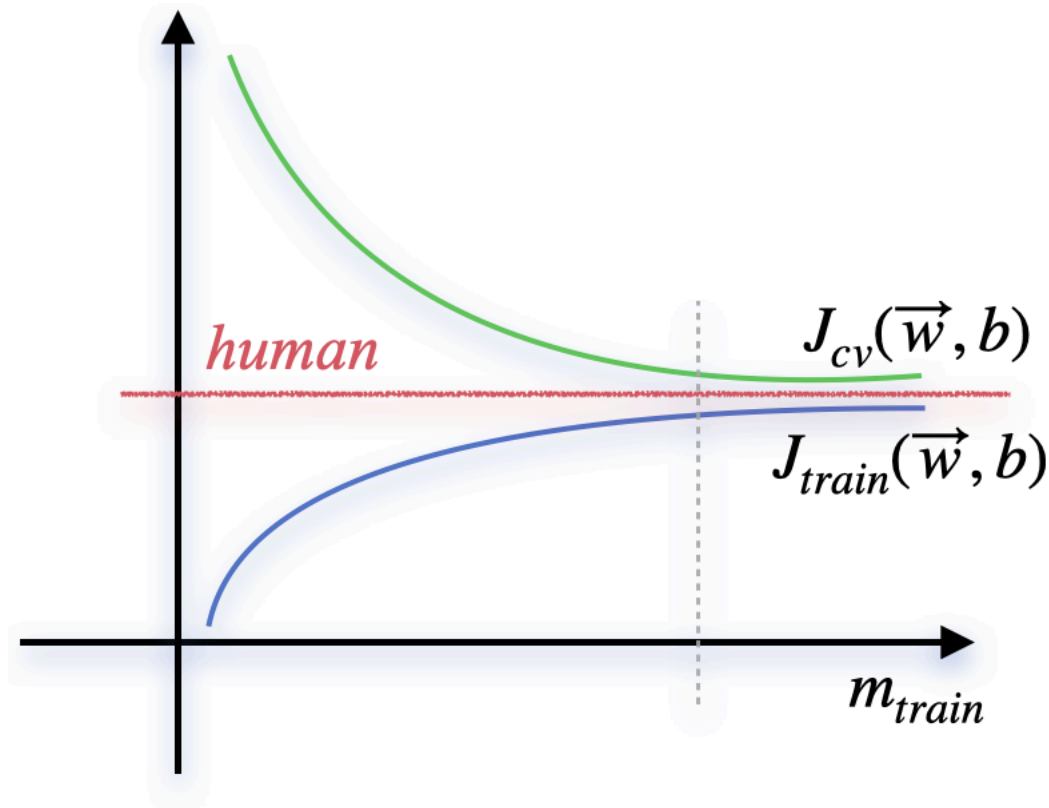
正如选择 $\vec{w}$ 和 b 一样, 正则化的参数 λ 也可以利用 Cross Validation Set 选择合适的值。



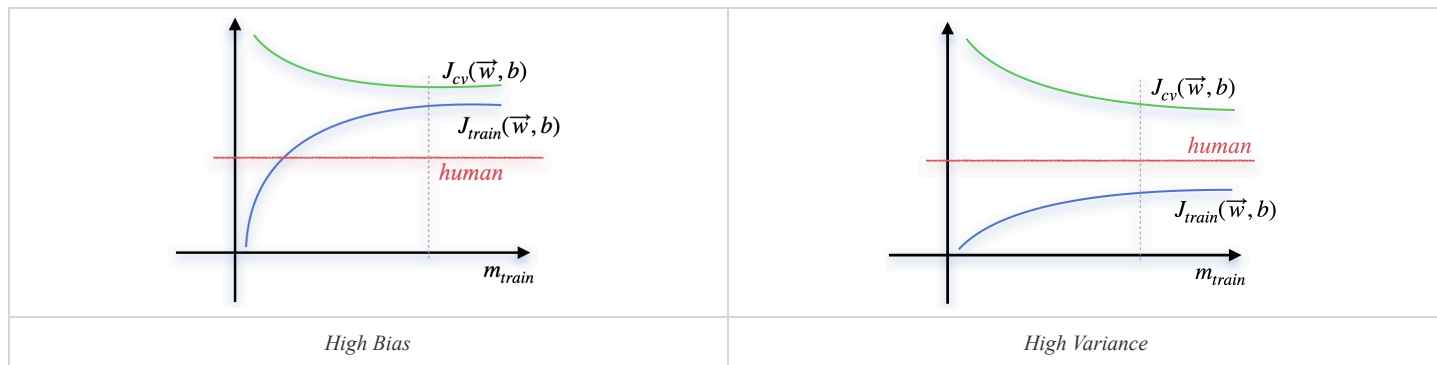
左侧部分（high variance 区域）， J_{train} 会很低， J_{cv} 会很高，并且 $J_{cv} \gg J_{train}$ ，过拟合、高方差。
 右侧部分（high bias 区域）， J_{train} 会很高， J_{cv} 也会很高，并且 $J_{cv} \approx J_{train}$ ，欠拟合、高偏差。

学习曲线

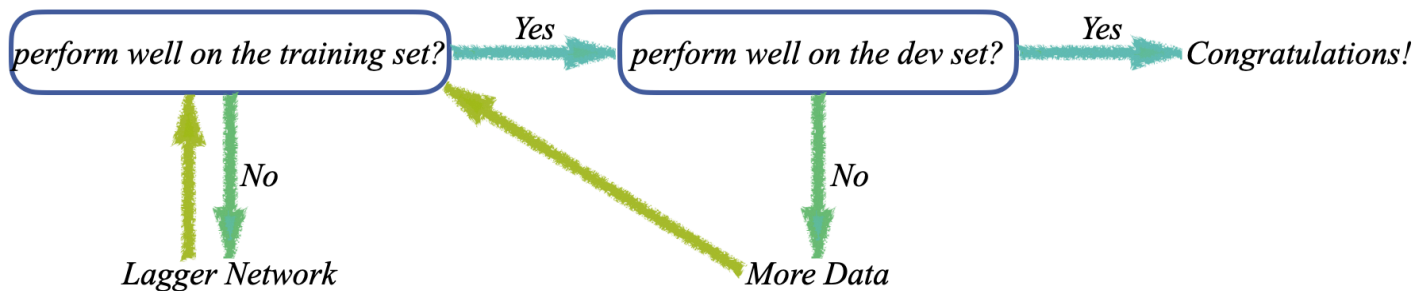
对于一个正常的模型来说，我们可以容易分析其学习曲线如下：



而对于欠拟合（左）与过拟合（右）的模型来说，其学习曲线如下所示：



在 Neural Network 中，有一个简单而有效的降低 Bias 和 Variance 的流程如下：



由以上流程也可知，对于 Neural Network 而言，正则化做得好，那么 Neural Network 越大越好，故以下是正则化后的 *TensorFlow* 代码：

```

model = Sequential(
    Dense(units=25, activation='relu', kernel_regularizer=L2(0.01))
    Dense(units=15, activation='relu', kernel_regularizer=L2(0.01))
    Dense(units=1, activation='sigmoid', kernel_regularizer=L2(0.01))
)
  
```

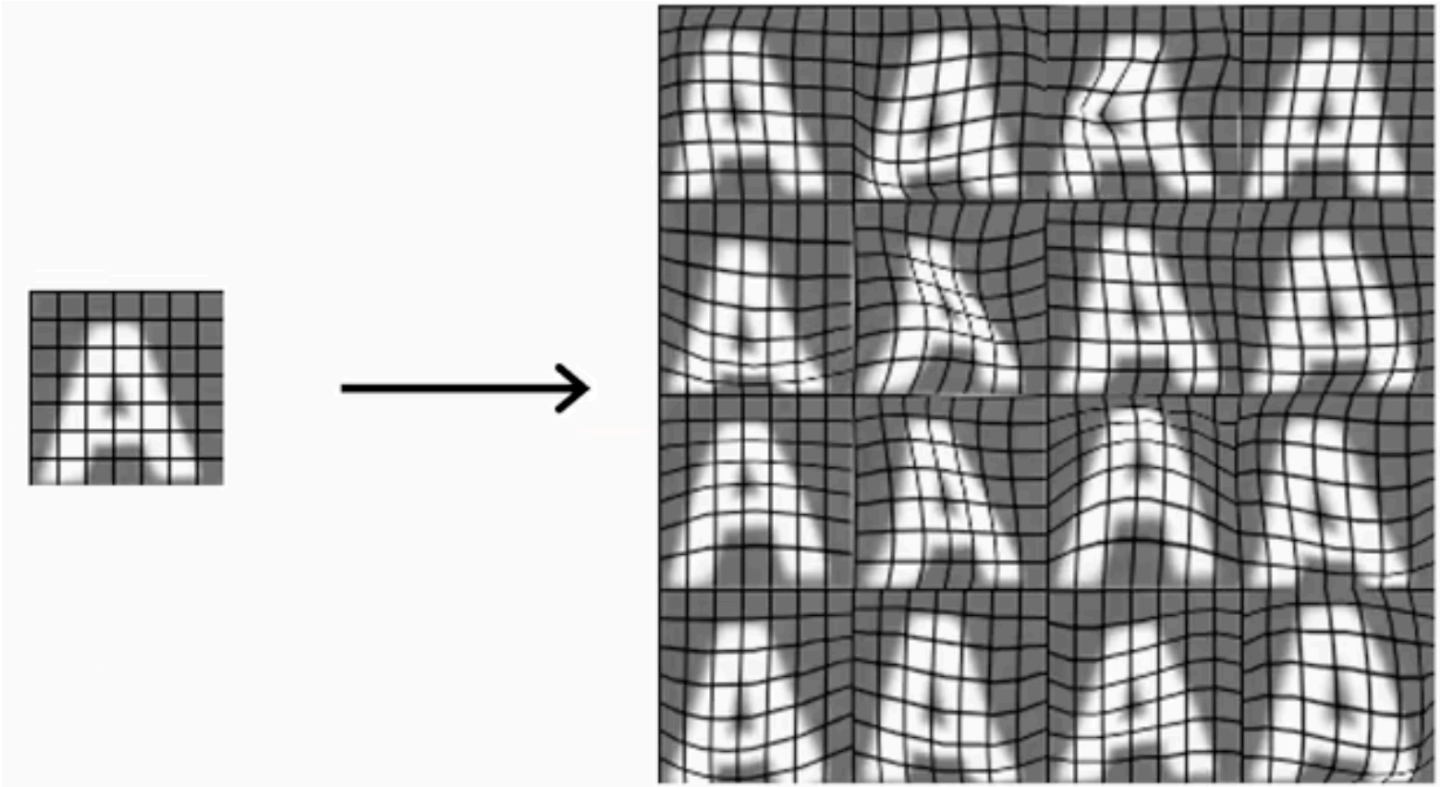
技巧补充

错误分析

以分类说明，错误分析指的是人工将模型分类错的 data 挑出来，分析其统一特征，比如垃圾邮件分类，可能就是把分错的邮件找出来，之后发现大部分是钓鱼邮件和地址异常导致的，此时再针对特定的邮件特点补充模型的特征或结构亦或是补充特定的 data。

Data Augmentation

Data Augmentation 指的就是对现有 data，尤其是图片，伸缩、旋转、平移、颜色等得到新的训练样本，从而扩充训练集。



但是某个标签的 data 不可由于 Data Augmentation 导致标签被修改，比如“猫”可以翻转还是得到“猫”，但“6”如果翻转就会得到“9”，不符合 Data Augmentation 的要求。

除了 Data Augmentation 之外，还有很多扩大训练集的方法，比如 Data Synthesis，即完全人工合成新的 data 而不是基于原有的 data 修改。

	
Real Data	Synthetic Data

迁移学习

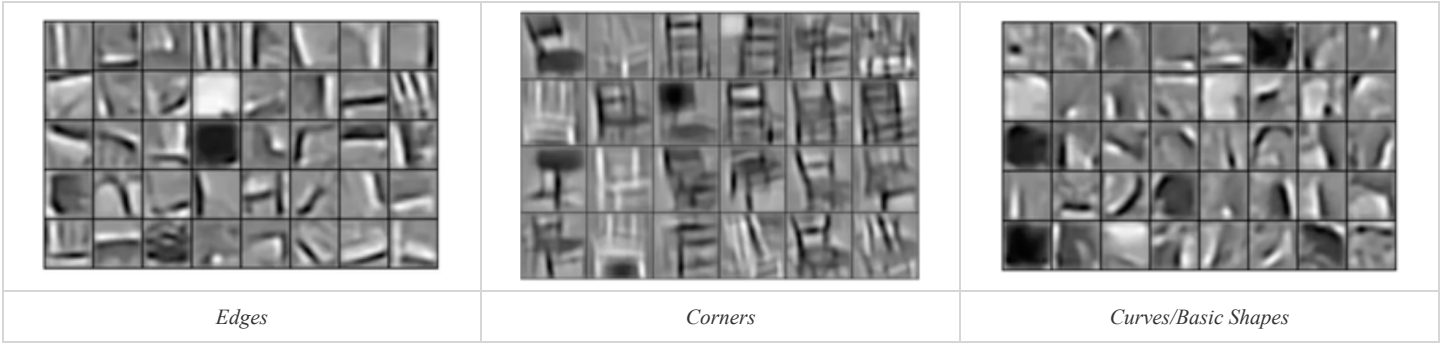
有时，采取了很多办法，训练集还很小，此时可以采取迁移学习，即先在一个无关的大型训练集上训练一个类似的模型，在得到一组较优参数后，再在该小训练集上做最后的训练，迁移的时候将前面的参数照抄，最后 Output Layer 的结构和参数再修正并训练。

在大型训练集上预训练的过程称作“监督预训练”，后面的再训练称作“微调”，后者可以只训练 Output Layer 的参数，固定其他的参数不动（小训练集），也可以全都训练（中训练集）。

The primary reason transfer learning is achievable is due to the hierarchical nature of Neural Networks.

The neurons in the initial layers learn basic image features, such as edges, lines, and so on. The neurons in the later layers then combine these earlier local features to ultimately form a complete image.

Therefore, a significant portion of the parameters in the initial hidden layers can be reused.



倾斜训练集 (Skewed Datasets)

倾斜训练集是指训练集中一个类别的样本远多于其他类别，比如某罕见病患比率只有 0.5%，然而你的模型有 1% 误判，此时如果有一个只输出 0 的模型，那么其好像比你的模型好，但是其实没有任何实际应用场景，为此无法通过误判率来评估模型。

为此，全面地评估模型就得引入混淆矩阵 (Confusion Matrix)，并在此进一步得到精准率、召回率和 F1 分数。

混淆矩阵 (Confusion Matrix)

	Predicted Positive	Predicted Negative
Actual Positive	TP (True Positive)	FN (False Negative)
Actual Negative	FP (False Positive)	TN (True Negative)

精准率 (Precision)

在所有模型预测为正的样本中真正为正的比率。

$$Precision = \frac{TP}{TP + FP}$$

召回率 (Recall) / 敏感性 (Sensitivity)

在所有实际为正的样本中，模型识别出来的比率。

$$Recall = \frac{TP}{TP + FN}$$

F1 分数 (F1 Score)

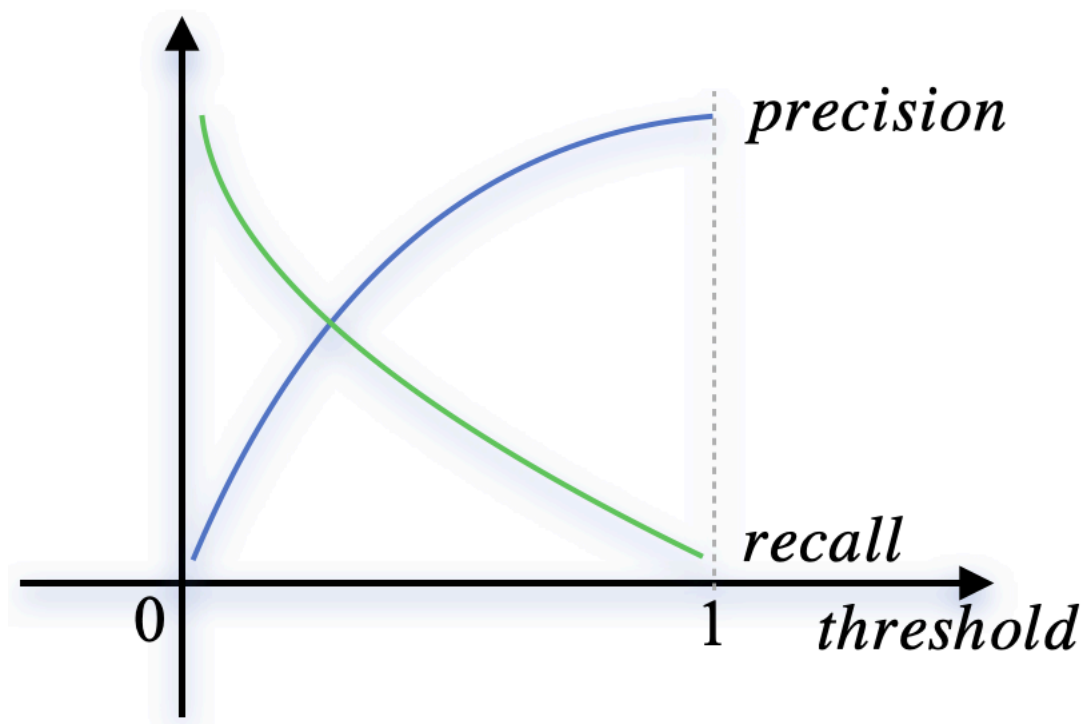
$\frac{2}{F_1\ Score} = \frac{1}{Precision} + \frac{1}{Recall}$ ，综合指标。

$$F_1\ Score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Precision-Recall 权衡

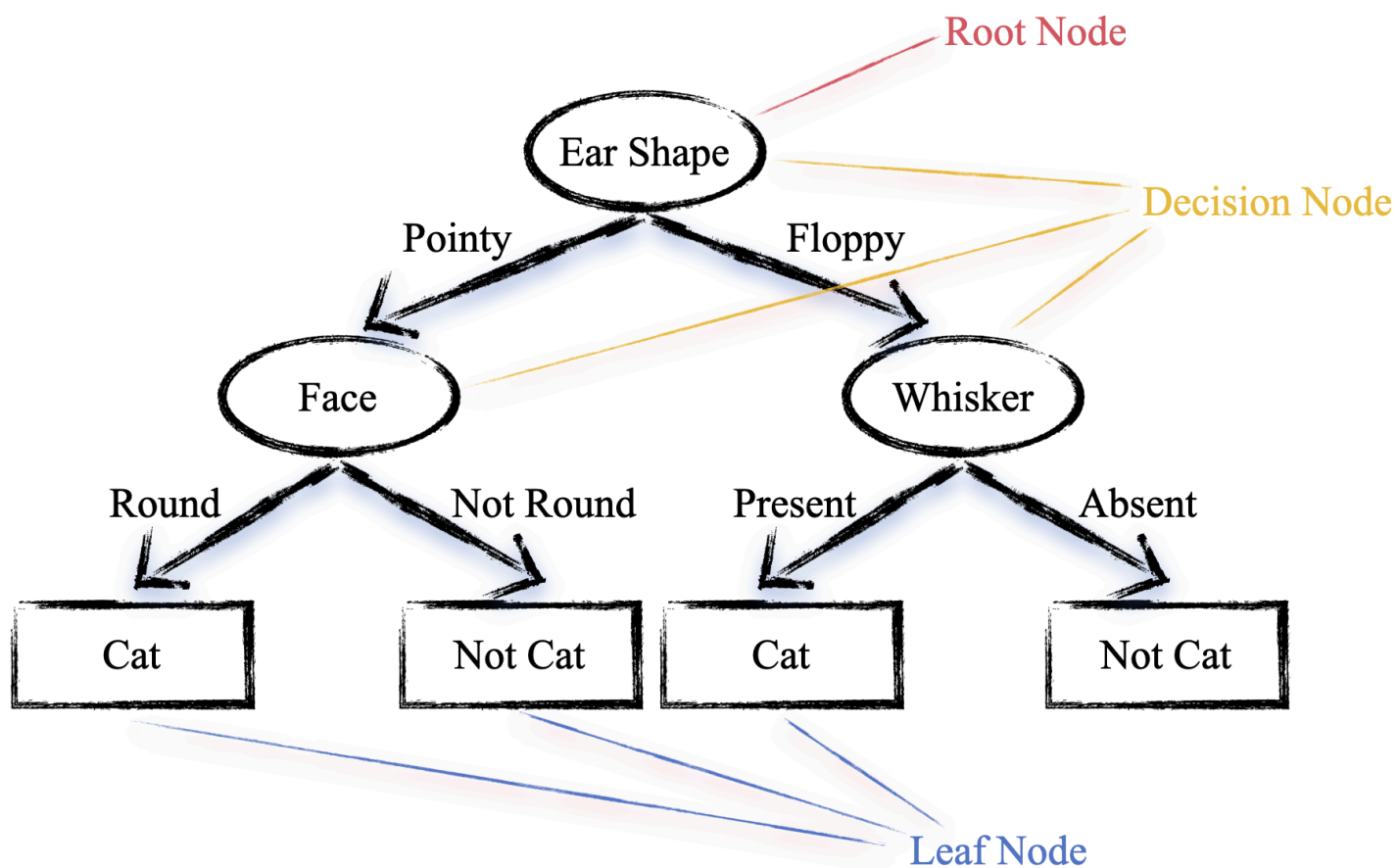
对于某些归类问题，降低阈值 (比如，从 0.5 降到 0.3)，模型会倾向将样本预测为正类，此时召回率会提高，但精准率会下降。当漏报 (FN) 的代价很高时，比如，检测病症、恐怖分子、欺诈行为，难以承担 FN 的代价就得尽可能提高召回率。（宁可错杀一人，不肯放过一个）

反之，提高阈值，模型将样本预测为正类的可能性降低，此时精准性可能提升，但召回率会下降，当误报 (FP) 的代价很高时，比如，垃圾邮件、推荐系统，难以承担 FP 的代价就得尽可能提高精准率。（你说他有病，丫就是有病）

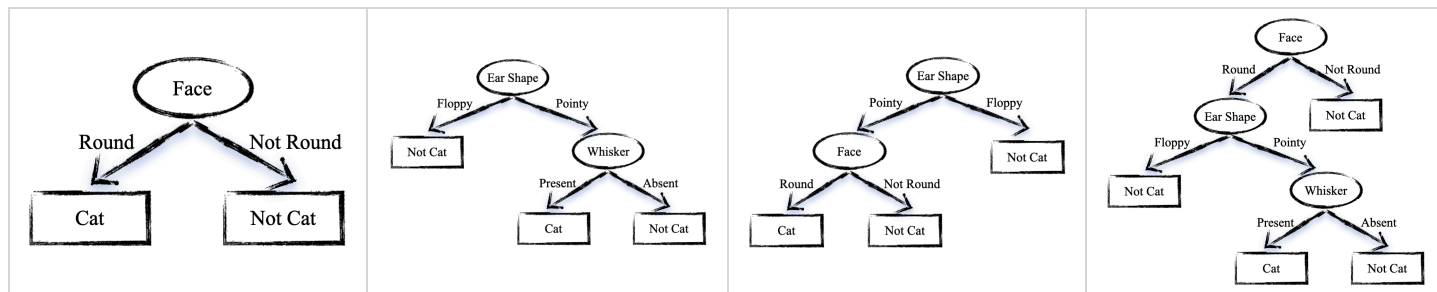


决策树

决策树是一种树状的模型，它通过一串基于特征的决策，将 **data** 从根节点开始逐步划分，最终到达叶子节点，得到最终的预测结果。



同一问题，决策树可以构造出的种类众多，决策树训练就是训练出结构最佳的树形，使得相应的指标最好，以下同对于猫归类问题而构造的不同形态的树：



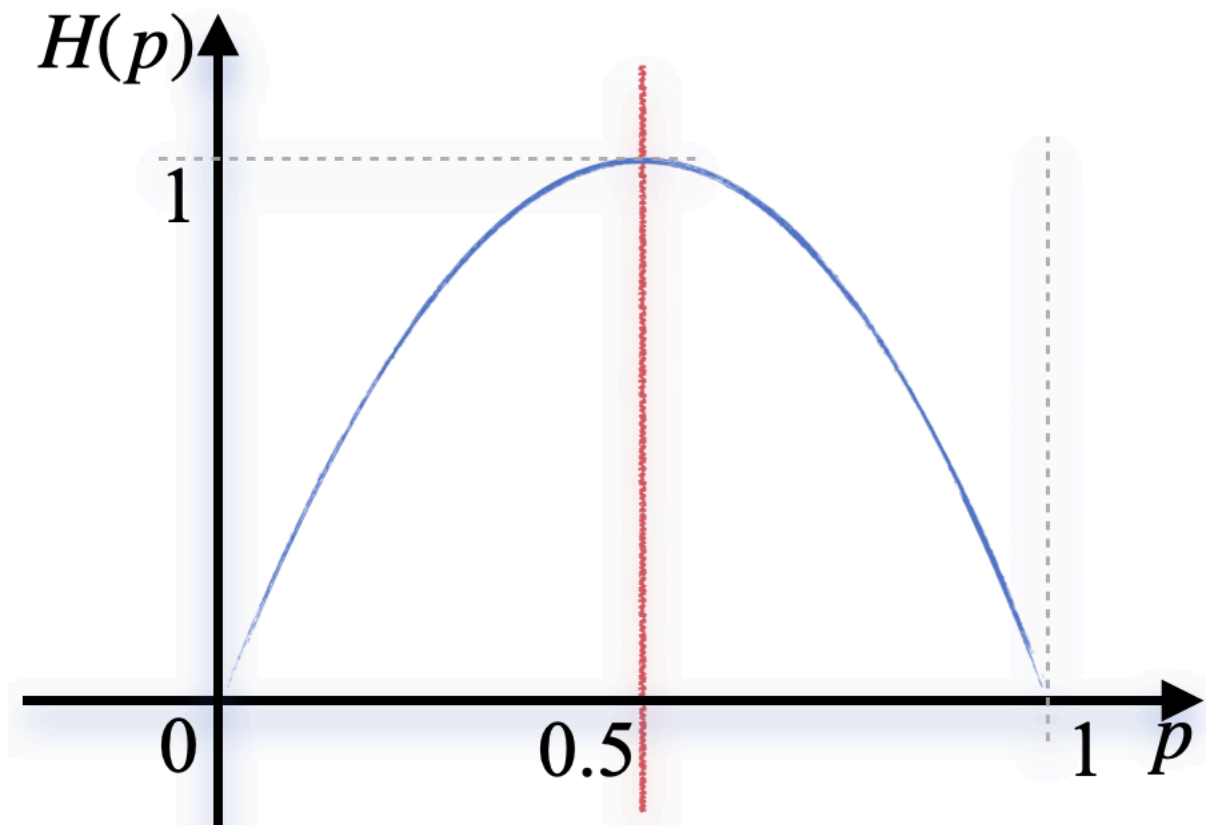
Recursion 二叉分裂

构造决策树的大体流程是 **recursion** + 贪心的二叉分裂的过程：

1. 从根节点开始：当前节点下的所有训练样本中
2. 选择最佳分裂点：在所有特征中，寻找一个最佳的特征（和它的最佳分裂阈值），能够将当前节点下的 **data** 划成两个子集，使得分裂后的子集**纯度**最高（即每个子集中的样本尽可能是同一类别）。
3. 创建子节点：由最佳分裂点将 **data** 分成两个子节点。
4. 迭代：对每个子节点实现上述过程，直到达到停止条件。
5. 生成叶子节点：当达到停止条件时，将当前节点标记为叶子节点，并赋予一个预测值。

熵 (Entropy)

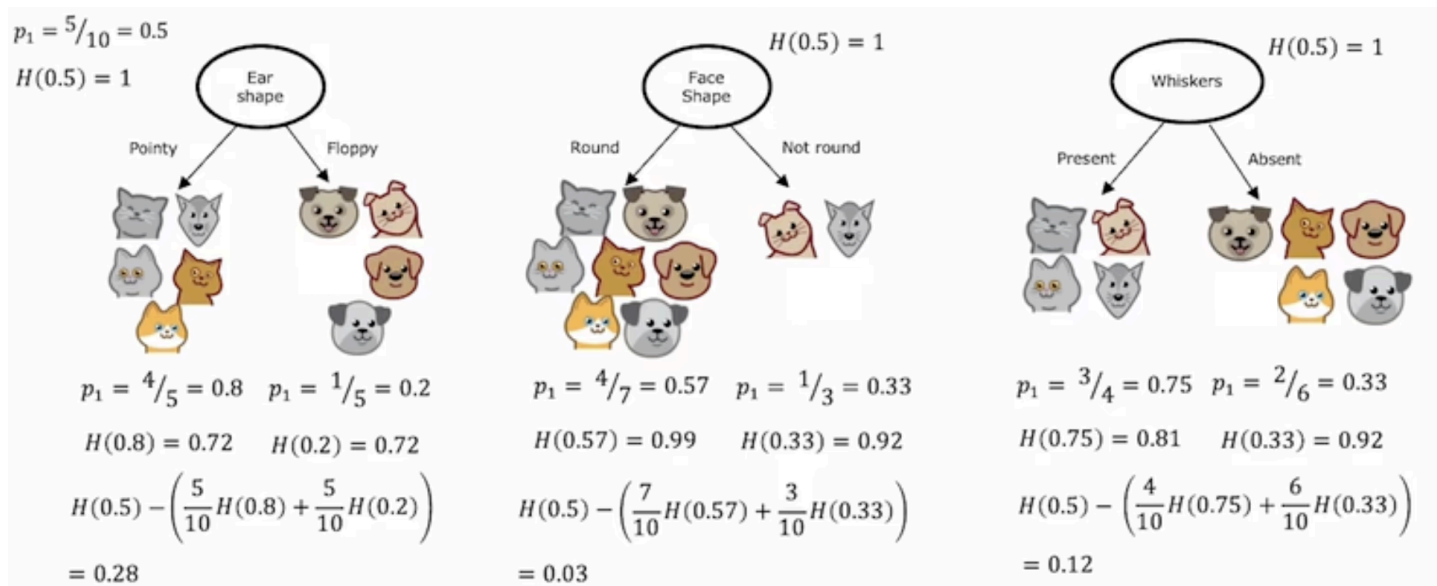
熵是反映 **data** 混乱性的指标，其与某类别占全体的比率的函数如下，熵越大，样本越不纯净：



其中 $H(p) = -p \log_2(p) - (1-p) \log_2(1-p)$

信息增益 (Information Gain)

信息增益 = 裂前的熵 - 裂后的加权平均熵。



通常定义一个集合中正例的比率是 p_1 ，则信息增益的公式可以写成（以根节点说明）：

$$IG = H(p_1^{root}) - (H(p_1^{left}) \cdot w^{left} + H(p_1^{right}) \cdot w^{right})$$

其中 w^{left} 指的是左侧子节点占总节点的比率。

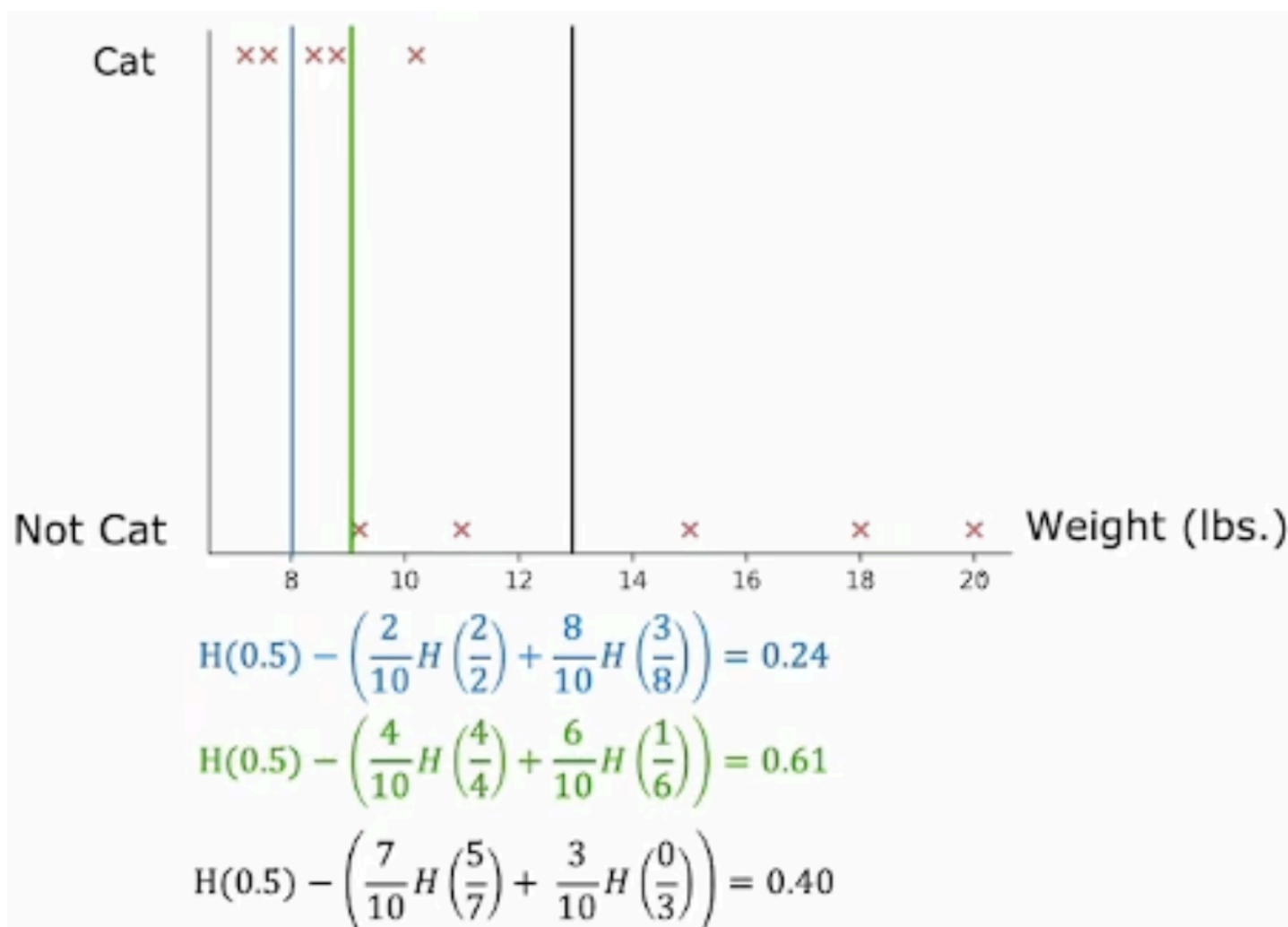
停止条件

常见的停止条件：

- 达到预设的最大深度 (**max_depth**)。
- 叶子节点包含的样本小于最小样本数 (**min_samples_leaf**)。
- 分裂后纯度提升小于某个阈值。
- 节点内所有样本都是同一类别（完全纯净）。
- 没有特征可以分裂。

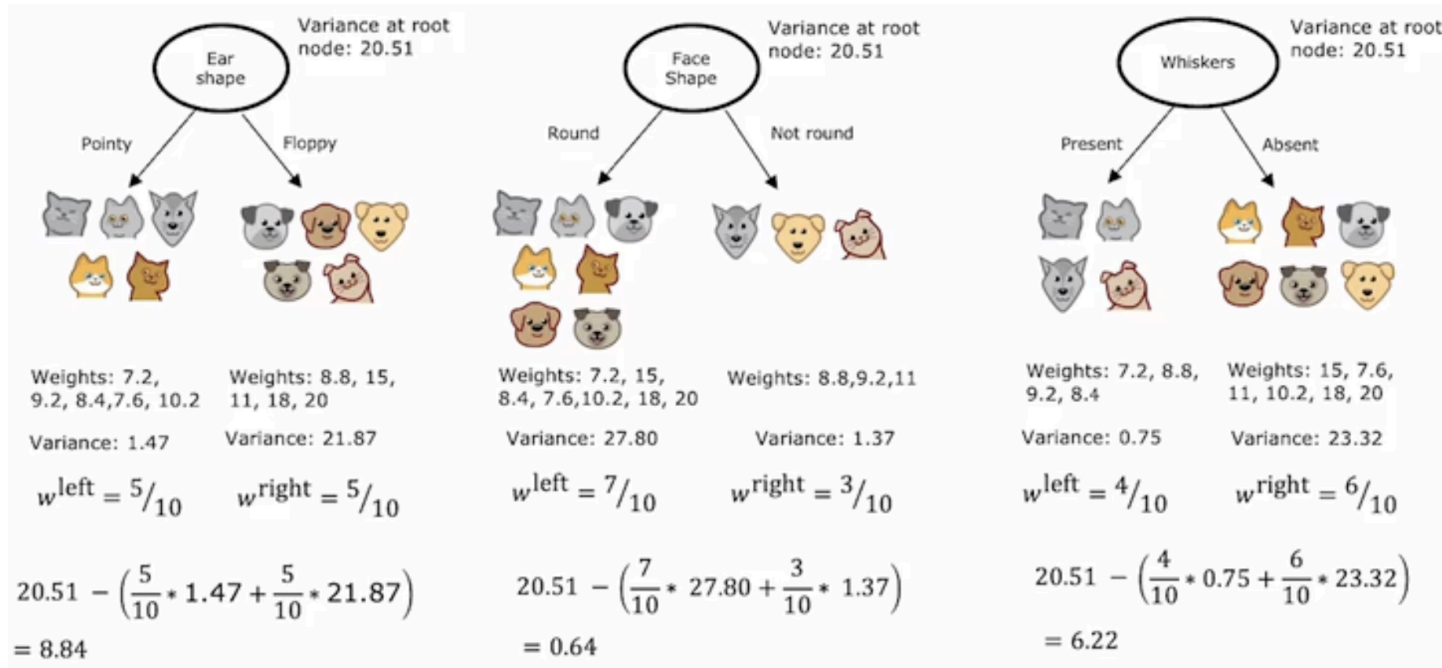
Continuous Variable

决策树不仅可以对离散值分类，也可以像 **Neural Network** 一样搞定 **Continuous Variable**。



回归问题

回归问题树称为回归树，其不再用熵作为划开子树的标准，通常选择能最小化裂开后子节点内 MSE 的特征和阈值。



集成树 (Ensemble Methods)

放回抽样

在原有训练集上利用放回抽样可以模拟出众多新的训练集，新的训练集可能不包括原有训练集的所有 data，也可能有重复，但是无伤大雅。

随机森林

在利用放回抽样得到众多新的训练集后，每个训练集可能构造出不同的决策树，由此些决策树组合成的集合称作随机森林。

不过有时放回抽样训练得到的树形态可能非常相似，所以通常限制节点选择特征时只能在大特征集合 (n) 的随机子集 ($k = \sqrt{n}$) 中选择。

XGBoost 优化

优化放回抽样时，可能不再等概率抽取原训练集中的样本，而是针对性地偏向抽取已训练出来的决策树在原训练集中预测不佳的样本。具体优化细节不在此介绍，其中优化好的就有 XGBoost 。

```
from xgboost import XGBClassifier # from xgboost import XGBRegressor

model = XGBClassifier() # model = XGBRegressor()

model.fit(x_train, y_train)
y_pred = model.predict(x_test)
```

1. 标准术语集

- Neural Network: 神经网络
 - Input Layer: 输入层
 - Hidden Layer: 隐藏层
 - Output Layer: 输出层
- Machine Learning: 机器学习
- Data Augmentation: 数据增强
- Recursion: 递归/迭代
- Continuous Variable: 连续值
- MSE: 均方误差

